We Wasted a Year Migrating 300+ Spark Jobs — Here's How Docker Made Sure It Never Happens Again

Anurag Sharma 7 min read · Mar 30, 2026

We went from EMR → Databricks → EMR again — and paid for it with nearly a year of engineering time. What looked like a simple platform switch turned into months of rework again after three years of debugging and chasing subtle differences across environments. The frustrating part wasn't just the effort — it was how little of it actually moved the business forward. We were re-solving the same problems, just in a different runtime.

That is when we realized this was not a one-time mistake. It was a systemic issue with how our Spark jobs were built and deployed. Not because Spark changed. Not because our data changed. But because our *environment did*.

That is when it clicked: the real problem was not the platform we chose — it was how tightly our code was tied to it.

So we made a hard pivot.

We containerized every Spark job using Docker — so the next time we migrate, it is not a rewrite. It is just a redeploy.

Let us deep dive.

The Real Problem: Platform Lock-In Isn't Always Obvious

Most teams think vendor lock-in comes from proprietary formats or APIs.

We thought we were safe — we used open formats like Parquet and Delta.

We were wrong.

The real lock-in came from:

- Runtime behavior differences
- Dependency management quirks
- Cluster configuration assumptions
- Platform-specific optimizations

These subtle differences turned what should have been a simple migration into a **6-month engineering drain**.

The Honeymoon Phase: From EMR to Databricks

Like many teams, we started on **AWS EMR**. It was reliable for our batch Spark workloads, integrated seamlessly with S3 and the broader AWS ecosystem, and felt cost-effective for periodic ETL jobs.

Then we discovered **Databricks**. The promise was irresistible:

- Optimized Spark runtime (with Photon engine)
- Unified notebooks for collaboration
- Easier autoscaling and job management
- Built-in features like Delta Lake that reduce boilerplate code

We migrated our pipelines. For the **past three years**, everything has run smoothly. Developer productivity soared. Complex transformations felt simpler. The platform "just worked."

But as our data volumes grew and jobs multiplied (we now manage **300+ Spark jobs**), the bills started hurting badly.

The Harsh Reality: When Databricks Bills Hit Revenue

Databricks' **DBU (Databricks Unit)** pricing adds a premium on top of underlying compute costs. For many organizations — especially smaller analytics companies like ours — this premium becomes painful at scale. Independent comparisons consistently show Databricks costing **2–4x more** than raw EMR for similar batch workloads, especially when not heavily optimized.

We were not alone. Many teams report that while Databricks shines for bursty, interactive, or ML-heavy workloads, pure cost-sensitive batch processing often favors simpler managed Spark options like EMR (with Spot instances, right-sizing, and transient clusters).

Faced with real revenue pressure, we made the tough call: **migrate everything back to AWS EMR**.

The Painful Migration: 6 Months of Rework

This was not a simple switch. Differences in:

- Runtime behavior and configurations
- Dependency management
- Cluster tuning
- Integration nuances (e.g., S3 access patterns, metastore handling, custom libraries)

We had to rewrite, retest, and re-optimize hundreds of jobs. What should have been weeks of work dragged into **six months**. Combined with the original move to Databricks, we effectively lost nearly a **full year** of productive engineering time. 1 Year => 6 Months EMR to DB, then again 6 Months of DB to EMR.

The financial hit was twofold: high Databricks costs during the "good" years, followed by a heavy engineering investment to return to a lower baseline. It was a classic vendor lock-in trap — not through proprietary data formats (we stayed on open standards like Parquet and Delta), but through platform-specific optimizations and workflows that made code non-portable.

The Decision: Containerize Spark Code in Docker Images

We refused to repeat this cycle. Our goal became clear: Build Spark applications that are **truly portable** across any Docker-friendly environment

— whether it's EMR, self-managed Spark on EC2/EKS, Kubernetes (Spark-on-K8s), another cloud, or even on-prem.

The solution? Package every Spark job (or group of related jobs) inside Docker images.

Here's why this approach made sense for us:

1. **Environment Consistency** — The same Spark version, Python/Java dependencies, custom JARs, configurations, and even OS-level libraries travel with the code. No more "it works on Databricks but fails on EMR" surprises.
2. **True Lift-and-Shift** — Future migrations become far simpler. Submit the same Docker image to EMR's Docker support, run it via Spark-on-Kubernetes, or deploy to any container-orchestrated environment. No deep code rewrites.
3. **Reproducibility** — Local development, CI/CD testing, and production all use identical images. "Works on my machine" disappears.
4. **Dependency Isolation** — Python packages, JAR conflicts, and version mismatches are contained. Upgrading a library? Rebuild and test the image once.
5. **Lightweight & Optimized Code** — During our return to EMR, we already rewrote jobs to be more efficient (better partitioning, caching, Adaptive

Query Execution, etc.). Containerizing locked in those optimizations.

AWS EMR itself supports running Spark applications inside Docker containers (via YARN container runtime configurations), making this a natural fit.

How We're Implementing It

- **Multi-stage Dockerfiles**: Keep images lean—one stage for building dependencies, another for the runtime Spark image.
- **Base Spark Images**: Start from official apache/spark images and layer our job-specific code, configs, and requirements.
- **Job-Specific or Shared Images**: For common libraries, we use shared base images. High-customization jobs get their own.
- **Orchestration**: Airflow, MWAA, or EMR Steps submit the containers with proper resource requests.
- **CI/CD**: Every code change triggers image builds, tests (including local Spark clusters in Docker), and pushes to a registry (ECR or equivalent).
- **Configuration as Code**: Spark configs live in the image or are passed at runtime, avoiding hard-coded platform differences.

This adds some upfront effort in Dockerfile maintenance, but it pays off massively in reduced migration toil.

Why This Changes Everything

1. True Portability

Run the same image on:

- EMR
- Kubernetes (Spark-on-K8s)
- EKS
- Any Docker-friendly platform

No rewrites.

2. Zero Environment Drift

No more:

- "Works on Databricks but fails on EMR."
- Version mismatch debugging

3. Reproducibility

- Local = CI = Production
- Deterministic builds

4. Dependency Isolation

Upgrade safely without breaking other jobs.

5. Migration Becomes Boring (That's Good)

Future platform changes = infrastructure problem, not code rewrite.

Is This the Right Thing? (Honest Assessment)

Yes — **for our context**, this is a smart, pragmatic move.

Pros:

- Protects against future cost shocks or feature gaps.
- Reduces long-term engineering debt from platform churn.
- Aligns with industry trends toward containerized data pipelines and Spark-on-Kubernetes for maximum portability.
- Works especially well for batch ETL workloads.

Potential Drawbacks & Trade-offs:

- Slightly more operational complexity (image building, registry management, scanning for vulnerabilities).
- Not a silver bullet — Databricks-specific features (e.g., advanced Delta Live Tables, optimized autoscaling magic) may still require some adaptation.
- Requires the target platform to support Docker (most modern ones do: EMR, EKS, self-managed K8s, etc.).
- Initial investment in standardizing images across 300+ jobs.

The core problem you faced was **genuine**. Many companies experience sticker shock with Databricks for high-volume batch work, and migrations between managed Spark platforms are notoriously non-trivial due to subtle runtime and configuration differences — even when using open-source Spark under the hood.

Containerization doesn't eliminate all differences (e.g., cluster management, monitoring, security policies), but it dramatically reduces the biggest pain point: making your *application code* portable.

Lessons Learned & Advice for Other Teams

- **Evaluate Total Cost of Ownership Early:** Developer velocity on Databricks is real, but model long-term costs carefully, especially for predictable batch jobs.
- **Stay Close to Open Standards:** Avoid over-relying on proprietary APIs where possible.
- **Test Portability Continuously:** Run jobs locally in Docker + on multiple platforms during development.
- **Combine with Other Best Practices:** Use Spot instances, right-sized transient clusters, and aggressive code optimization across platforms.
- **Consider Spark-on-Kubernetes** as a potential long-term target for even greater control and multi-cloud flexibility.

If you are a small-to-medium analytics team running mostly batch Spark pipelines, strongly consider containerizing early. It won't make platform choices irrelevant, but it makes them far less painful.

We are now much more confident about future moves — whether back to a managed service, to Kubernetes, or even hybrid setups. Our Spark jobs are no longer tied to one vendor's runtime quirks.

What We Would Do Differently (If Starting Today)

1. Containerize from Day 1
2. Continuously test on multiple platforms
3. Avoid deep coupling with platform-specific features
4. Design for portability — not convenience

Final Thoughts

> The goal isn't to avoid choosing platforms. The goal is to avoid being trapped by them.

Docker didn't eliminate differences between platforms.

But it turned migrations from a **rewrite problem** into a **deployment problem.**

And that changes everything.

Databricks | Data Engineering | Docker | Spark | Big Data

Written by Anurag Sharma

96 followers · 3 following

Data Engineering Specialist with 10+ exp. Passionate about optimizing pipelines, data lineage, and Spark performance and sharing insights to empower data pros!

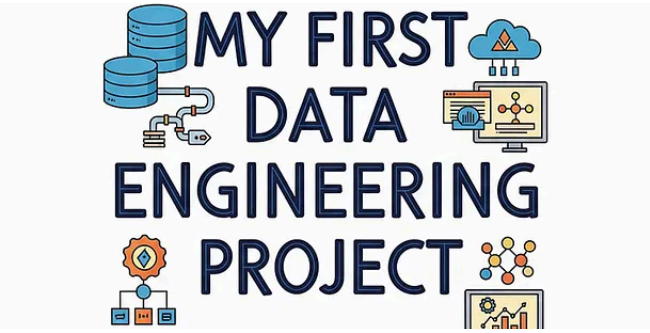
Edit profile

No responses yet

Anurag Sharma him/he

What are your thoughts?

More from Anurag Sharma



Anurag Sharma

My First Data Engineering Project: Phase 2— Migrating DataStage...

The second chapter of my data engineering journey! The second phase of a...

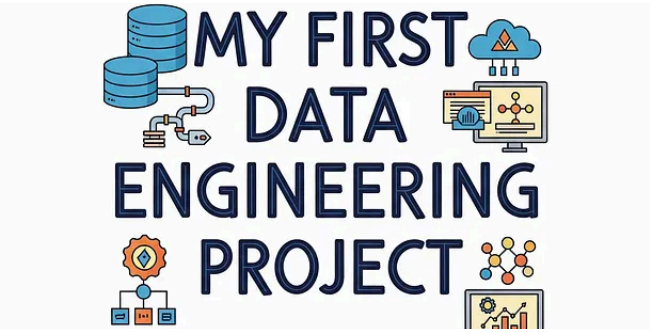
Jun 11, 2025 1 1

Anurag Sharma

How Important It Has Become to Build a Pet Project or Product-...

The New Gold Standard for Data Engineering Interviews Now

1d ago 21



Anurag Sharma

My First Data Engineering Project: Phase 1—Migrating a Massive Da...

Anurag Sharma

The Essential Skill Stack: Foundational Knowledge Every...

Welcome to the first chapter of my data engineering journey! This blog dives into a...

Jun 10, 2025 1

Welcome to Data Engineering 101.

Feb 23 21

See all from Anurag Sharma